

How noise_cancel.py Works

A walkthrough of the free, local alternative to Krisp — real-time mic noise cancellation using DeepFilterNet and BlackHole.

1. The Big Picture

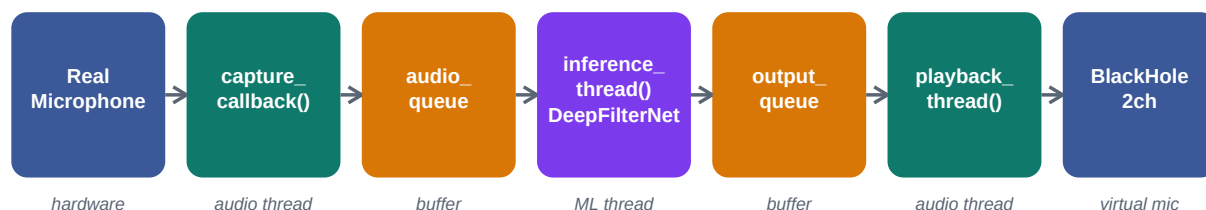
Krisp works by sitting between your real microphone and your meeting app: it intercepts the raw audio, runs it through a noise-removal model, and feeds the cleaned-up result to the app as if it were a microphone. This script does exactly the same job, using two free, open building blocks instead of Krisp's closed system:

- **DeepFilterNet** — an open-source deep learning model that has learned what human speech sounds like versus background noise, and subtracts the noise.
- **BlackHole 2ch** — a free macOS "virtual audio device." To the operating system, it looks exactly like a real microphone, except its input is whatever another program writes into it. This is the trick that lets your cleaned audio show up as a selectable mic in Zoom, Discord, Meet, etc.

The script's job is to be the middleman: continuously read from your real mic, push every chunk of audio through DeepFilterNet, and write the result into BlackHole as fast as the audio is coming in. "Real-time" here means the round trip — mic in, model, virtual mic out — has to happen in roughly the same amount of time it takes to speak that audio, or you'll hear lag or stutter.

2. The Pipeline at a Glance

Three threads run concurrently, handed off through two thread-safe queues:



Audio flows left to right through two in-memory queues. Queues are what let three independent threads — capture, inference, and playback — each work at their own pace without blocking each other.

3. Why Three Threads and Two Queues?

The core design problem this script solves is a speed mismatch. Audio capture and playback have to happen at a very strict, steady pace — your sound card delivers exactly 960 samples every 20 milliseconds, like clockwork, whether your code is ready or not. Running a neural network, on the other hand, takes a variable amount of time depending on your CPU load at that moment.

If the script tried to do everything in one place — capture a chunk, run the model on it, then write it out, all before grabbing the next chunk — any slowdown in the model would cause the microphone callback to fall behind, and macOS's audio system would treat that as a glitch (an "xrun"). The fix is to decouple the three stages with thread-safe queues:

Thread	Role	Speed requirement
Main / capture (<code>sd.InputStream callback</code>)	Grabs ~20ms of mic audio at a time, drops it into <code>audio_queue</code> , returns immediately.	Must be near-instant — this runs on a real-time audio callback.
inference_thread	Pulls raw audio from <code>audio_queue</code> , runs it through <code>DeepFilterNet</code> , pushes the cleaned audio to <code>output_queue</code> .	Can take variable time — this is where the ML model does its work.
playback_thread	Pulls cleaned audio from <code>output_queue</code> and writes it to the <code>BlackHole</code> virtual output device.	Needs steady pacing — writes silence if <code>output_queue</code> runs dry, to avoid underflow.
print_stats	Every 5 seconds, prints a status line with processed/dropped counts and queue sizes.	Cosmetic — for your visibility only.

A Python `queue.Queue` is what makes this safe: multiple threads can call `.put()` and `.get()` on it at the same time without corrupting each other's data — Python handles the locking internally. Each queue here is capped at 50 chunks (about 1 second of audio at 20ms blocks) so that if something stalls, the buffer fills up and starts shedding old audio rather than growing forever and introducing unbounded lag.

4. Walking Through the Code, Section by Section

4.1 — Setup: arguments, imports, device discovery

The script starts with `argparse`, giving you command-line flags (`--mic`, `--out`, `--atten`, `--blocksize`) instead of hardcoded values. The imports are wrapped in `try/except` blocks that print a plain-English instruction (e.g. "Run: `pip install sounddevice`") rather than a raw stack trace if a dependency is missing — this is just defensive UX, not part of the audio logic.

`find_blackhole()` loops over every audio device the OS reports and looks for one whose name contains "BlackHole" and which accepts output (i.e. you can write audio into it). This is how the script auto-detects where to send the cleaned audio without you having to know its device index.

4.2 — Loading the model: `init_df()`

`init_df()` from the `df` (DeepFilterNet) package loads the pretrained neural network weights and returns three things: the `model` itself, a `df_state` object that holds configuration like the required sample rate, and a `config` object (unused here, hence the underscore). Critically, `df_state.sr()` tells the script `DeepFilterNet` was trained on 48kHz audio — the script then opens both the input and output audio streams at exactly that rate, because feeding the model audio at the wrong sample rate would make it think pitches and noise patterns are different from what it learned.

4.3 — The capture callback (real-time thread)

```
def capture_callback(indata, frames, time_info, status):
    try:
        audio_queue.put_nowait(indata[:, 0].copy())
    except queue.Full:
        stats["dropped"] += 1
```

`sounddevice` calls this function automatically, on its own dedicated real-time thread, every time it has a fresh block of microphone audio ready (every 20ms by default). `indata` arrives as a 2D array shaped (frames, channels); since the mic is opened as mono, `indata[:, 0]` grabs the single channel as a flat array. `.copy()` matters here — without it, the array would reference a buffer that `sounddevice` reuses on the next callback, corrupting the data before the inference thread gets to read it.

`put_nowait()` is used instead of `put()` deliberately: this function must return almost instantly or it risks glitching the audio hardware. If the queue is already full (50 chunks backed up), it raises `queue.Full` immediately rather than blocking, and the `except` clause just counts it as a dropped chunk and moves on.

4.4 — The inference thread (the fix you needed)

```
def inference_thread():
    while not stop_event.is_set():
        try:
            chunk = audio_queue.get(timeout=0.1)
        except queue.Empty:
            continue
```

```

audio_np = chunk.astype(np.float32)
audio_2d = audio_np[np.newaxis, :] # shape (1, N)
audio_tensor = torch.from_numpy(audio_2d) # <-- the fix

try:
    enhanced = enhance(model, df_state, audio_tensor, atten_lim_db=args.atten)
    result = enhanced[0].detach().cpu().numpy()
except Exception as e:
    result = audio_np # pass through unfiltered on error

try:
    output_queue.put_nowait(result)
except queue.Full:
    stats["dropped"] += 1 # <-- the other fix
stats["processed"] += 1

```

This is the heart of the script, and where the original crash lived. Step by step:

1. `audio_queue.get(timeout=0.1)` blocks for up to 0.1 seconds waiting for a chunk; if nothing arrives it loops back and checks `stop_event` again, which is how Ctrl+C eventually reaches this thread.
2. `DeepFilterNet's enhance()` is a PyTorch function under the hood, so it expects a `torch.Tensor` — not a plain NumPy array — shaped (channels, samples), which is why the code adds a new axis (`np.newaxis`) to turn a flat array of N samples into a (1, N) tensor.
3. **This is exactly where your error happened.** The original code passed `audio_2d` (still a NumPy array) straight into `enhance()`, which internally calls `torch.nn.functional.pad()` — a function that only accepts tensors. `torch.from_numpy(audio_2d)` converts it without copying memory, which is both correct and fast.
4. `enhance()` returns a tensor; `.detach()` strips away PyTorch's gradient-tracking bookkeeping (irrelevant here since this is inference, not training), `.cpu()` ensures it's on regular memory, and `.numpy()` converts it back to a plain array so the rest of the pipeline — which is all NumPy — can use it.
5. The outer `try/except Exception` is a safety net: if the model ever throws an error on a particular chunk, the script falls back to passing the unfiltered audio through rather than going silent. **This is what masked your original bug** — the `TypeError` was being caught here, but the fallback line itself had no protection against a full queue, so it crashed the thread anyway, just one line later and with a more confusing second traceback.
6. `atten_lim_db` is the attenuation ceiling — how many decibels of noise reduction the model is allowed to apply at most. Your `--atten` flag controls this directly.

4.5 — The playback thread

```

def playback_thread():
    with sd.OutputStream(device=output_device, samplerate=SAMPLE_RATE,
                        channels=1, dtype="float32", blocksize=BLOCK) as out_stream:
        while not stop_event.is_set():
            try:
                chunk = output_queue.get(timeout=0.1)

```

```
except queue.Empty:
    out_stream.write(np.zeros(BLOCK, dtype=np.float32))
    continue
out_stream.write(chunk.astype(np.float32))
```

This opens a dedicated output stream targeting the BlackHole device and continuously writes whatever cleaned audio is available. The interesting detail is the `except queue.Empty` branch: rather than just waiting silently, it explicitly writes a block of zeros (silence) to the stream. Audio output devices generally expect a continuous, gapless stream of samples — if you don't write anything for a stretch, you get clicks, pops, or the stream may stall entirely. Writing silence keeps the stream healthy even if the model temporarily falls behind.

4.6 — Stats, signal handling, and shutdown

`print_stats()` runs on its own thread purely for your visibility, printing processed/dropped counts and current queue depths every 5 seconds — this is how you noticed the problem in the first place (`queue_out` climbing while `dropped` stayed at 0, then the crash). `signal.signal(signal.SIGINT, handle_sigint)` intercepts Ctrl+C so that instead of Python's default abrupt kill, the script sets `stop_event`, which all three background threads are polling for — they each notice it within their 0.1-second timeout and exit their loops cleanly, and the main thread then `.join()`s them (waits for them to finish) before printing "Goodbye."

5. Diagnosing the Crash You Hit

Your log showed processed counts climbing steadily with zero drops, then a sudden exception that killed the inference thread, after which drops exploded. Here's the chain of cause and effect:

Symptom in your log	Root cause	Fix
<code>TypeError: pad(): argument 'input' ... must be Tensor, not numpy.ndarray</code>	<code>enhance()</code> is a PyTorch function. It was being handed a raw NumPy array instead of a <code>torch.Tensor</code> .	Wrap the chunk with <code>torch.from_numpy(...)</code> before calling <code>enhance()</code> .
<code>queue.Full</code> raised inside the except block, which killed the inference thread	The error-fallback line tried <code>output_queue.put_nowait(...)</code> with no protection — if the queue was already full, this raised an unhandled exception.	Wrap every <code>put_nowait()</code> call in its own <code>try/except queue.Full</code> and drop the chunk instead of crashing.
dropped count exploding after the crash	Once <code>inference_thread</code> died, nothing drained <code>audio_queue</code> anymore, so <code>capture_callback</code> started dropping everything.	A side-effect of the above — fixing the crash fixes this automatically.

6. Why This Replaces Krisp

Functionally, both tools do the same job — model-based speech/noise separation, fed into the OS as a virtual microphone — but the building blocks differ:

	Krisp	This script
Noise removal engine	Proprietary neural net (cloud-trained, closed source)	DeepFilterNet — open-source neural net, runs fully locally
Cost	Subscription after free tier	Free
Where it runs	Locally, but model is closed	Locally, fully inspectable
Output mechanism	Built-in virtual audio device	BlackHole 2ch (open-source virtual audio driver)
Customization	Fixed behavior	You control attenuation (<code>--atten</code>), block size, device selection, etc.

Because both DeepFilterNet and BlackHole are open-source and run entirely on your machine, there's no per-minute cloud processing, no account, and no subscription — the trade-off is that you're now the one responsible for keeping the glue script (this file) running correctly, which is exactly the kind of bug you just hit.

7. Practical Notes Going Forward

- **--atten** controls aggressiveness. Higher values remove more noise but can introduce more artifacts in your voice; if voices start sounding robotic or underwater, try lowering it.
- **queue_out climbing steadily** (as in your log, before the crash) is a useful early-warning sign — it means inference is keeping up with capture, but playback isn't draining the output queue fast enough, or inference is slightly faster than playback's consumption rate. Worth watching if you see it again.
- **The errors counter** added in the fix will now print a warning periodically if the model keeps failing on certain chunks, instead of silently degrading to pass-through audio forever.
- **--blocksize** trades latency for stability. Smaller blocks (e.g. 480 = 10ms) reduce the delay between speaking and being heard, but give the model and threads less room before they fall behind; larger blocks are more forgiving but add lag.

Generated as a companion explainer for noise_cancel.py — based on the working, patched version of the script.